

Informe final del Proyecto Programado: NextShop

Curso: Programación Web II
Proyecto: NextShop
Integrantes: Edgardo Mora, Oscar Marín
Fecha: Junio de 2026
Documento: Memoria técnica final

Control del documento

Elemento	Valor
Archivo principal	docs/informe-final.md
Evidencias visuales	docs/evidencias/
Documentos de apoyo	docs/ProyectoProgramado.md , docs/inventario-productos-imagenes.md , docs/catalogo-visual-productos.md , docs/fuente-imagenes-productos.md , docs/catalogo-imagenes-productos.md
Estado	Versión final para entrega académica

Ficha técnica del proyecto

Elemento	Valor
Proyecto	NextShop
Tipo de sistema	Tienda en línea académica
Framework	Spring Boot
Motor de plantillas	Thymeleaf
Arquitectura	MVC por capas

Lenguaje principal	Java
Persistencia actual	Repositorios en memoria
Persistencia prevista	Preparado para JPA
Recursos estáticos	CSS e imágenes desde <code>src/main/resources/static/</code>
Control de versiones	Git
Build	Maven
Estado	Funcional

Índice

1. Resumen ejecutivo
 2. Introducción
 3. Planteamiento del problema
 4. Objetivos del proyecto
 5. Alcance funcional
 6. Marco conceptual aplicado
 7. Arquitectura general de NextShop
 8. Modelo de datos en memoria
 9. Gestión de productos y catálogo visual
 10. Módulo de imágenes de productos
 11. Interfaz de usuario con Thymeleaf
 12. Gestión de sesiones, login y perfiles
 13. Carrito de compras
 14. Módulo administrativo
 15. Validaciones y reglas de negocio
 16. Pruebas y evidencias
 17. Control de versiones
 18. Limitaciones actuales
 19. Recomendaciones y trabajo futuro
 20. Conclusiones
 21. Anexos
-

1. Resumen ejecutivo

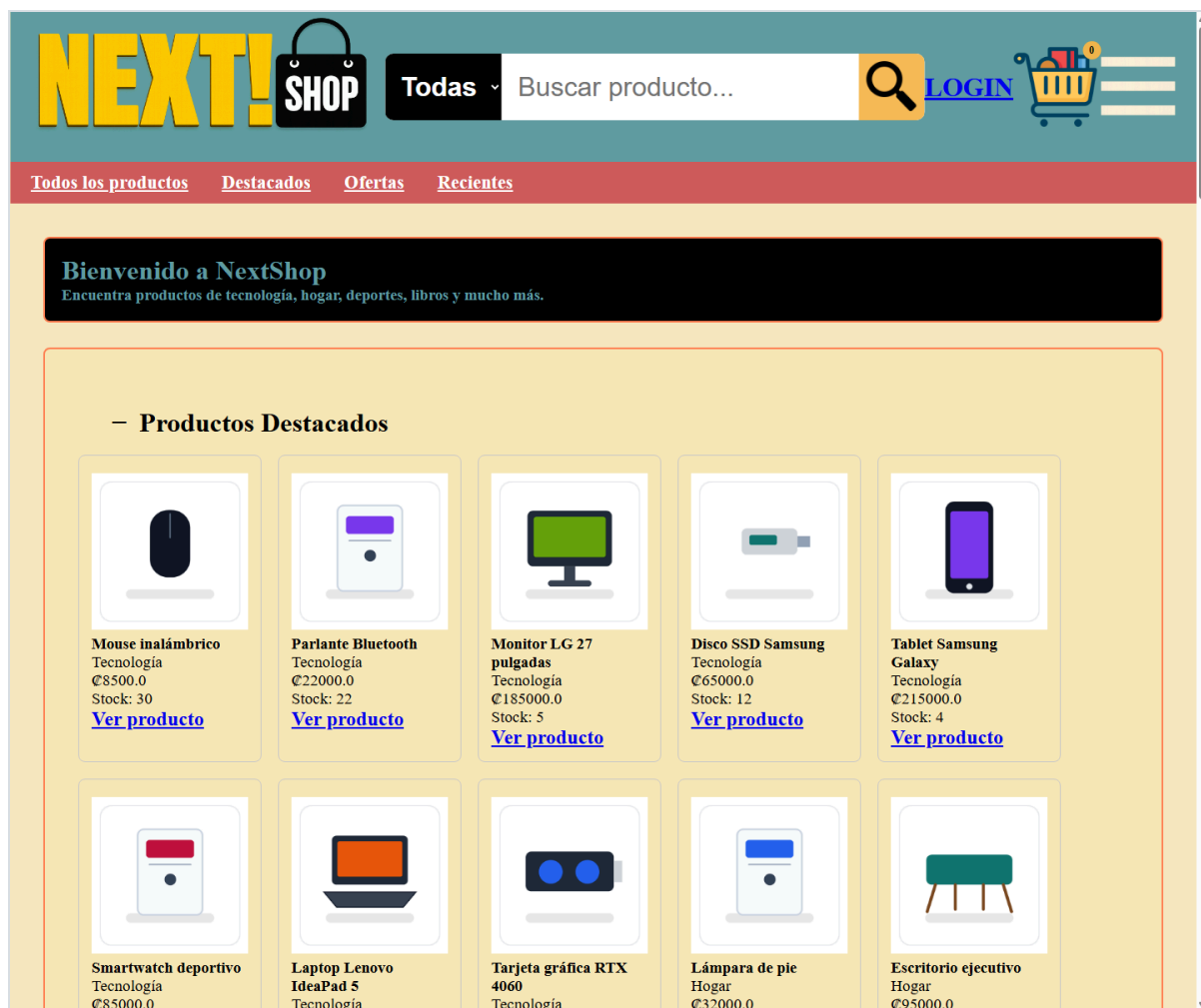
NextShop es una aplicación web desarrollada con Spring Boot y Thymeleaf que simula una tienda en línea. El sistema permite consultar productos, ver detalles, iniciar sesión, agregar productos al carrito y administrar información del catálogo desde módulos internos.

La versión documentada trabaja con repositorios en memoria. Esta decisión permite validar la arquitectura, las reglas de negocio y la interacción entre controladores, servicios, repositorios y vistas antes de una migración completa a base de datos con JPA.

Uno de los avances principales del proyecto fue la integración del catálogo visual de productos. Se auditaron 99 registros, se definieron 87 rutas PNG únicas y se conectó cada producto con su imagen mediante `imagePath`.

Esta implementación corresponde al uso del patrón MVC, recursos estáticos y separación por capas estudiados durante el curso.

Evidencia principal



Home con productos e imágenes

La captura muestra la página principal renderizando productos reales del repositorio con imágenes cargadas desde `imagePath`. Evidencia la integración entre repositorio, servicio, controlador, vista Thymeleaf y recursos estáticos.

2. Introducción

El proyecto NextShop fue desarrollado como una solución web académica para aplicar los conceptos centrales de Programación Web II. La aplicación representa una tienda en línea con navegación pública, autenticación de usuarios, carrito de compras y módulos administrativos.

La implementación se organizó en capas para mantener responsabilidades claras. Los controladores atienden solicitudes HTTP, los servicios aplican reglas de negocio, los repositorios simulan el acceso a datos y las vistas Thymeleaf presentan la información al usuario.

El proyecto también incorpora un módulo visual de productos. Las imágenes se almacenan como archivos PNG dentro de `static/images/products/` y se referencian desde cada producto mediante una ruta pública.

Evidencias relacionadas

- `docs/evidencias/01-home-productos-imagenes.png` : muestra la página principal con productos e imágenes.
 - `docs/evidencias/02-detalle-producto-imagen.png` : muestra un producto individual con su imagen asociada.
-

3. Planteamiento del problema

Una tienda en línea necesita organizar productos, categorías, usuarios, inventario, carrito y recursos visuales de manera coherente. Si estas responsabilidades se mezclan, el sistema se vuelve difícil de mantener y de extender.

NextShop resuelve este problema mediante una estructura MVC por capas. La información de productos se administra desde repositorios en memoria, la lógica se concentra en servicios y las vistas consumen los datos preparados por los controladores.

El reto más visible en esta etapa fue reemplazar imágenes genéricas por imágenes específicas de producto sin modificar la base funcional del sistema. Para ello se documentó el catálogo, se prepararon imágenes PNG y se conectaron mediante `imagePath`.

Esta solución aplica separación de responsabilidades, renderizado dinámico y uso de recursos estáticos.

4. Objetivos del proyecto

Objetivo general

Desarrollar una aplicación web de comercio electrónico llamada NextShop que permita gestionar y visualizar productos, usuarios, carrito de compras y recursos estáticos aplicando los conceptos de Programación Web II.

Objetivos específicos

- Implementar una arquitectura MVC organizada por capas.
- Construir vistas dinámicas con Thymeleaf.
- Simular persistencia mediante repositorios en memoria.
- Permitir consulta, búsqueda y detalle de productos.
- Gestionar sesiones de usuario para clientes y administradores.
- Integrar imágenes de productos mediante rutas `imagePath`.
- Validar el funcionamiento con compilación, rutas HTTP y evidencias visuales.

Evidencias relacionadas

- `docs/evidencias/08-build-success.png` : confirma compilación exitosa.
- `docs/evidencias/09-git-status.png` : confirma el estado del repositorio al momento de generar evidencias.

5. Alcance funcional

La versión actual de NextShop incluye funcionalidades públicas, autenticadas y administrativas.

Funcionalidades públicas

- Visualización de productos en la página principal.
- Búsqueda de productos por categoría y nombre.
- Visualización del detalle de un producto.
- Registro e inicio de sesión.

Funcionalidades autenticadas

- Acceso al carrito de compras.
- Agregado de productos al carrito.
- Visualización de productos agregados.
- Consulta de cuenta de usuario.

Funcionalidades administrativas

- Acceso al panel administrativo.
- Listado y búsqueda de productos.
- Creación y edición de productos.
- Activación y desactivación de productos.
- Gestión básica de clientes.

Fuera del alcance actual

- Base de datos externa como fuente principal.
- Pasarela de pagos real.
- Uso de BLOB para imágenes.
- Despliegue productivo.
- Configuración completa de Spring Security.

Esta sección evidencia el alcance práctico del sistema y evita afirmar funcionalidades no implementadas.

6. Marco conceptual aplicado

NextShop aplica los conceptos del curso desde una implementación concreta, no como un resumen teórico.

Arquitectura MVC

Los controladores reciben solicitudes HTTP, los servicios procesan reglas de negocio, los repositorios entregan datos y Thymeleaf renderiza las vistas.

Spring Boot

El proyecto utiliza Spring Boot para ejecutar la aplicación, registrar componentes y servir recursos web.

Thymeleaf

Las vistas usan expresiones como `th:src`, `th:text` y `th:href` para mostrar datos enviados por los controladores.

Repositorios en memoria

Los datos se almacenan temporalmente en listas y estructuras Java. Esta solución facilita la validación funcional antes de una migración a JPA.

Recursos estáticos

Las imágenes de productos se sirven desde `src/main/resources/static/images/products/` mediante rutas públicas como `/images/products/laptop-lenovo-ideapad-5.png`.

Esta implementación corresponde al uso del patrón MVC, plantillas del lado servidor y recursos estáticos vistos durante el curso.

7. Arquitectura general de NextShop

La arquitectura del proyecto está organizada por capas:

Capa	Responsabilidad	Ejemplos
Controller	Atiende solicitudes HTTP y selecciona vistas o redirecciones	HomeController , ProductDetailsController , ShoppingCartController , AdminProductsController
Service	Aplica reglas de negocio y validaciones	ProductService , ClientService , ShoppingCartService , InventoryService
Repository	Simula acceso a datos en memoria	ProductRepository , ClientRepository , CategoryRepository , InventoryRepository
Data / Model	Representa entidades del dominio	Product , Client , Category , Cart
Templates	Renderiza HTML con Thymeleaf	home.html , ProductDetails.html , ShoppingCart.html
Static	Expone CSS, imágenes y recursos públicos	images/products/ , css/

Esta organización permite ubicar cada cambio en una capa específica. Por ejemplo, la conexión de imágenes se realizó en `ProductRepository` y las vistas ya consumían `product.imagePath` , por lo que no fue necesario rediseñar la interfaz.

Evidencias relacionadas

- `docs/evidencias/06-productrepository-imagepath.png` : muestra productos con rutas reales.
- `docs/evidencias/07-carpeta-static-images-products.png` : muestra la carpeta de imágenes estáticas.

8. Modelo de datos en memoria

NextShop utiliza repositorios en memoria para manejar productos, categorías, clientes, inventario y carrito. La fuente real de productos de esta versión se encuentra en `ProductRepository.init()` .

Este enfoque permite ejecutar la aplicación sin depender de una base de datos externa. También facilita probar reglas de negocio, flujos de usuario y renderizado de vistas antes de una migración a JPA.

En el caso de productos, cada registro incluye información como identificador, SKU, nombre, descripción, precio, categoría, propiedades, estado, stock y ruta de imagen.

Esta implementación corresponde al uso de repositorios como abstracción de acceso a datos.

9. Gestión de productos y catálogo visual

El catálogo visual se construyó mediante una auditoría documental del inventario existente. El objetivo fue identificar los productos reales usados por el sistema y definir una imagen específica para cada producto único.

Documentos de apoyo:

- docs/inventario-productos-imagenes.md
- docs/catalogo-visual-productos.md
- docs/fuente-imagenes-productos.md
- docs/catalogo-imagenes-productos.md

Resultados del catálogo:

Indicador	Resultado
Registros auditados	99
Rutas PNG únicas	87
Duplicados controlados	12
Imágenes faltantes	0
Fallback conservado	productIcon.png

Los productos duplicados conservan la misma imagen cuando representan el mismo artículo. Esto evita inconsistencias visuales y mantiene trazabilidad con los registros originales.

Evidencias relacionadas

- docs/evidencias/01-home-productos-imagenes.png : muestra productos renderizados con imágenes.
- docs/evidencias/07-carpeta-static-images-products.png : muestra los PNG disponibles.

10. Módulo de imágenes de productos

El módulo de imágenes conecta cada producto con una ruta pública mediante el atributo `imagePath`. Las imágenes se encuentran en:

```
src/main/resources/static/images/products/
```

La ruta pública utilizada por la aplicación sigue este formato:

```
/images/products/nombre-del-producto.png
```

La integración se realizó actualizando los productos en `ProductRepository.init()`. No se modificaron nombres, categorías, precios, descripciones ni lógica de negocio. El fallback `productIcon.png` se conserva para casos futuros, pero los productos existentes ya usan rutas específicas.

Esta implementación evidencia el uso de recursos estáticos en Spring Boot y renderizado dinámico con Thymeleaf.

Evidencias



Ruta estática de imagen

La captura muestra una imagen servida directamente desde `/images/products/`. Confirma que Spring Boot expone correctamente los archivos ubicados en `static/images/products/`.

ProductRepository.java - imagePath reales

```

60:     toys = categoryRepository.findByCategoryName("Juguetes").orElseThrow();
61:     others = categoryRepository.findByCategoryName("Otros").orElseThrow();
62:
63:     // -----
64:     // TECNOLOGÍA - NORMALES
65:     // -----
66:     this.productList.add(new Product(1L, "USB-001", "Memoria USB 64 GB", "Dispositivo portátil de almacenamiento.", 6500, "/images/products/usb.png", technology, null, true));
67:     this.productList.add(new Product(2L, "CAM-001", "Webcam HD", "Cámara web para videollamadas.", 14500, "/images/products/webcam.png", technology, null, true));
68:     this.productList.add(new Product(3L, "CHR-001", "Cargador USB", "Cargador rápido para dispositivos móviles.", 9500, "/images/products/cargador.png", technology, null, true));
69:     this.productList.add(new Product(4L, "STD-001", "Base para monitor", "Soporte para monitor de escritorio.", 12500, "/images/products/base.png", technology, null, true));
70:     this.productList.add(new Product(5L, "PRN-001", "Impresora multifuncional", "Impresora para hogar y oficina.", 125000, "/images/products/impresora.png", technology, Arrays.asList(new Property("Conectividad", "WiFi"), new Property("Tecnología", "Inyección de tinta")), true));
71:     this.productList.add(new Product(6L, "RTR-001", "Router WiFi 6", "Router de alta velocidad.", 58000, "/images/products/router.png", technology, Arrays.asList(new Property("Velocidad", "1800 Mbps"), new Property("Banda", "Dual Band")), true));
72:
73:     // -----
74:     // TECNOLOGÍA - DESTACADOS
75:     // -----
76:     this.productList.add(new Product(7L, "MOU-001", "Mouse inalámbrico", "Mouse óptico para uso diario.", 8500, "/images/products/mouse.png", technology, null, true, 0, LocalDateTime.now().minusDays(90), true));
77:     this.productList.add(new Product(8L, "SPK-001", "Parlante Bluetooth", "Parlante inalámbrico portátil.", 22000, "/images/products/parlante.png", technology, null, true, 0, LocalDateTime.now().minusDays(60), true));
78:     this.productList.add(new Product(9L, "MON-001", "Monitor LG 27 pulgadas", "Monitor para productividad.", 185000, "/images/products/monitor.png", technology, Arrays.asList(new Property("Resolución", "2560x1440"), new Property("Frecuencia", "144 Hz")), true, 0, LocalDateTime.now().minusDays(12), true));
79:     this.productList.add(new Product(10L, "SSD-001", "Disco SSD Samsung", "Unidad de almacenamiento.", 65000, "/images/products/ssd.png", technology, Arrays.asList(new Property("Capacidad", "1 TB"), new Property("Formato", "M.2 NVMe")), true, 0, LocalDateTime.now().minusDays(12), true));
80:     this.productList.add(new Product(11L, "TAB-001", "Tablet Samsung Galaxy", "Tablet para entretenimiento.", 215000, "/images/products/tablet.png", technology, Arrays.asList(new Property("Pantalla", "10.4 pulgadas"), new Property("Almacenamiento", "128 GB")), true, 0, LocalDateTime.now().minusDays(12), true));
81:     this.productList.add(new Product(12L, "SMW-001", "Smartwatch deportivo", "Reloj inteligente.", 85000, "/images/products/smartwatch.png", technology, Arrays.asList(new Property("Pantalla", "AMOLED"), new Property("Resistencia", "5 ATM")), true, 0, LocalDateTime.now().minusDays(5), true));
82:
83:     // -----
84:     // TECNOLOGÍA - OFERTAS
85:     // -----
86:     this.productList.add(new Product(13L, "KEY-001", "Teclado mecánico", "Teclado mecánico para oficina y gaming.", 28500, "/images/products/teclado.png", technology, null, false, 15, LocalDateTime.now().minusDays(45), true));
87:     this.productList.add(new Product(14L, "AUD-001", "Auriculares inalámbricos", "Auriculares Bluetooth de uso diario.", 18000, "/images/products/auriculares.png", technology, null, false, 20, LocalDateTime.now().minusDays(25), true));
88:
89:     // -----
90:     // TECNOLOGÍA - DESTACADOS + OFERTA
91:     // -----

```

ProductRepository usando imagePath

La captura muestra productos con rutas como `/images/products/laptop-lenovo-ideapad-5.png`. Evidencia que el repositorio conecta datos del producto con recursos estáticos.

src/main/resources/static/images/products/

aspiradora-vertical.png
audifonos-inalambricos.png
balon-de-baloncesto.png
balon-de-futbol.png
banco-para-pesas.png
base-para-monitor.png
bicicleta-de-montana.png
bloques-de-construccion.png
boligrafo-azul.png
boligrafo-negro.png
bolso-de-mano.png
botella-deportiva.png
cafetera-domestica.png
calcetines-deportivos.png
calculadora-cientifica.png
calculadora-financiera.png
caminadora-electrica.png
camiseta-casual.png
canasta-de-ropa.png
cargador-usb-rapido.png
carpeta-plastica.png
carro-control-remoto.png
carro-de-juguete.png
casa-de-munecas.png
chaqueta-impermeable.png
chaqueta-ligera.png
cinta-adhesiva.png
cinturon-de-cuero.png
clean-code.png
colchon-ortopedico.png
cuaderno-universitario.png
cuerda-para-saltar.png
disco-ssd-samsung.png
don-quiote-de-la-mancha.png
drone-recreativo.png
dune-libro.png
el-hobbit.png
el-senor-de-los-anillos.png
enciclopedia-universal.png
escritorio-ejecutivo.png
fundacion-libro.png
gorra-ajustable.png
harry-potter-piedra-filosofal.png
impresora-multifuncional.png
introduccion-a-java.png
jeans-clasicos.png
juego-de-comedor.png
juego-de-cuchillos.png
juego-de-mesa.png
lampara-de-pie.png
laptop-lenovo-ideapad-5.png
lego-city.png
libro-1984.png
licuadora-domestica.png
mancuernas-ajustables.png
marcadores-de-colores.png
mat-para-yoga.png
memoria-usb-64gb.png
microondas-domestico.png
mochila-ejecutiva.png
monitor-lg-27-pulgadas.png
mouse-inalambrico.png
muneca.png
organizador-de-escritorio.png

Carpeta de imágenes estáticas

La captura muestra los archivos PNG disponibles, incluyendo `productIcon.png` . Evidencia que las rutas definidas apuntan a archivos físicos existentes.

11. Interfaz de usuario con Thymeleaf

Las vistas de NextShop se renderizan con Thymeleaf. Los controladores cargan datos en el `Model` y las plantillas los muestran mediante expresiones dinámicas.

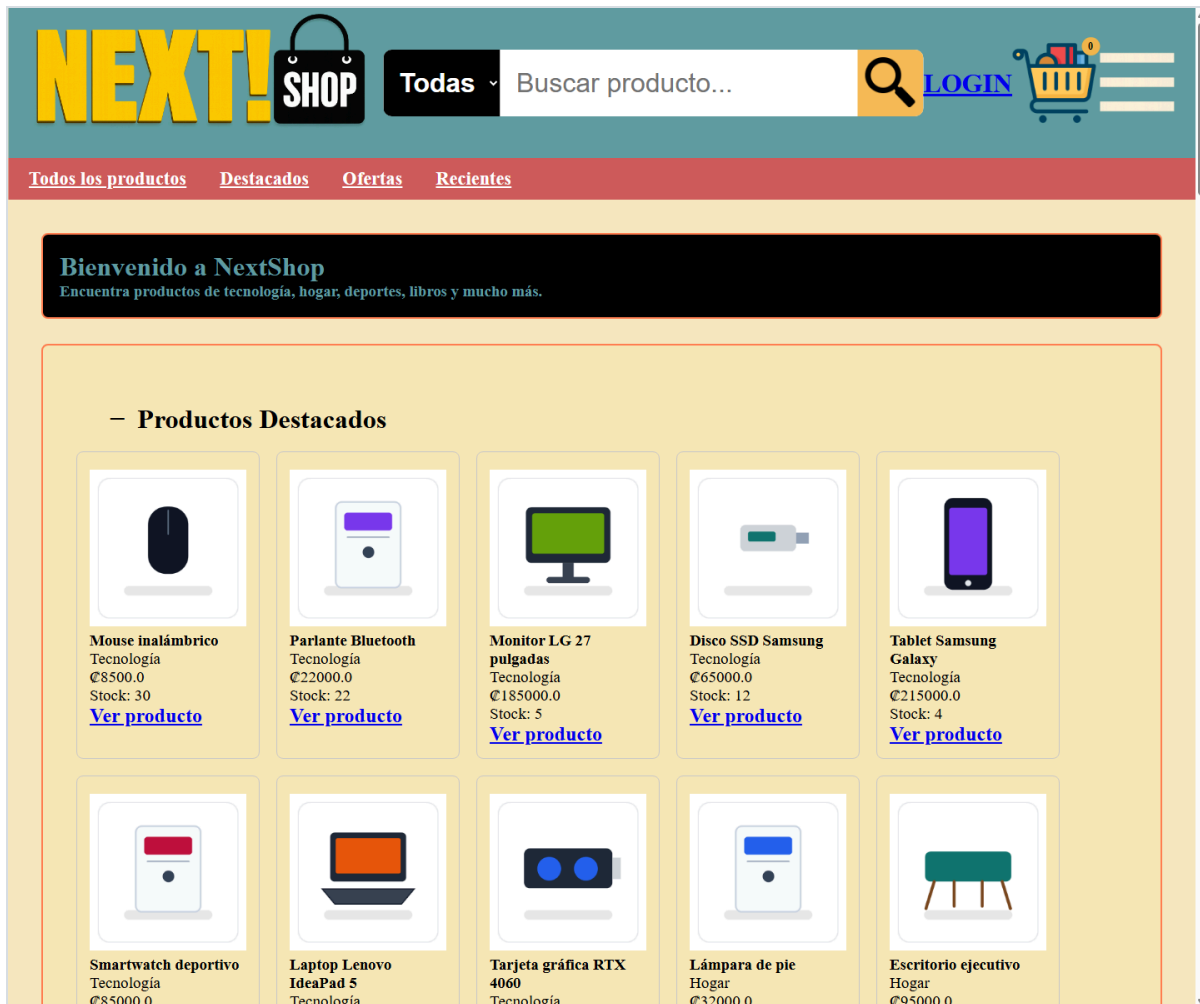
En el catálogo y detalle de productos, las imágenes se muestran usando `product.imagePath`. Esto permite que la vista no conozca el nombre físico de cada archivo; solo consume la ruta definida en el producto.

Vistas relacionadas:

- `src/main/resources/templates/pages/home.html`
- `src/main/resources/templates/pages/ProductDetails.html`
- `src/main/resources/templates/pages/ShoppingCart.html`
- `src/main/resources/templates/fragments/ProductData.html`
- `src/main/resources/templates/managment/AdminProducts.html`

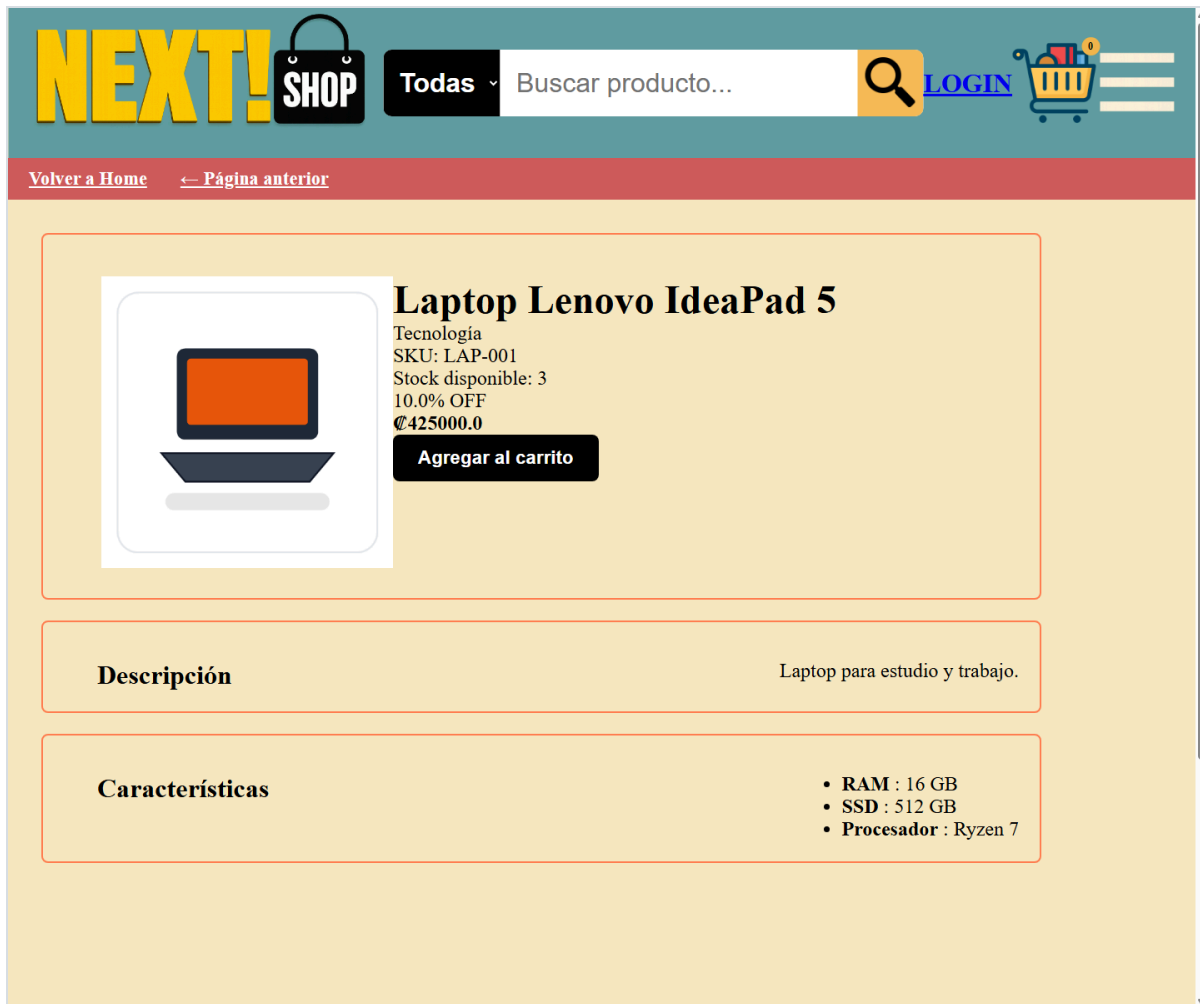
Esta implementación corresponde al renderizado de vistas del lado servidor visto en el curso.

Evidencias



Home con productos e imágenes

La captura muestra tarjetas de producto con nombre, categoría, precio, stock e imagen. Evidencia el uso de Thymeleaf para presentar datos del backend.



Detalle de producto con imagen

La captura muestra el detalle de un producto individual con su imagen asociada. Evidencia la relación entre ruta dinámica y vista de detalle.

12. Gestión de sesiones, login y perfiles

NextShop maneja usuarios mediante sesión HTTP. El sistema diferencia entre clientes y administradores, y utiliza esa información para permitir o restringir funcionalidades.

El login valida correo, contraseña y estado activo del usuario. Una vez autenticado, el cliente queda disponible en sesión para operaciones como acceder al carrito o agregar productos.

Esta implementación corresponde al manejo de estado con `HttpSession`, un concepto fundamental en aplicaciones web tradicionales.

Evidencia relacionada

- `docs/evidencias/04-carrito-imagen-producto.png`: muestra un producto agregado al carrito en una sesión de cliente.
-

13. Carrito de compras

El carrito de compras está asociado al cliente autenticado. Para agregar un producto, el sistema valida que el usuario tenga sesión activa, que el producto exista y que cuente con stock disponible.

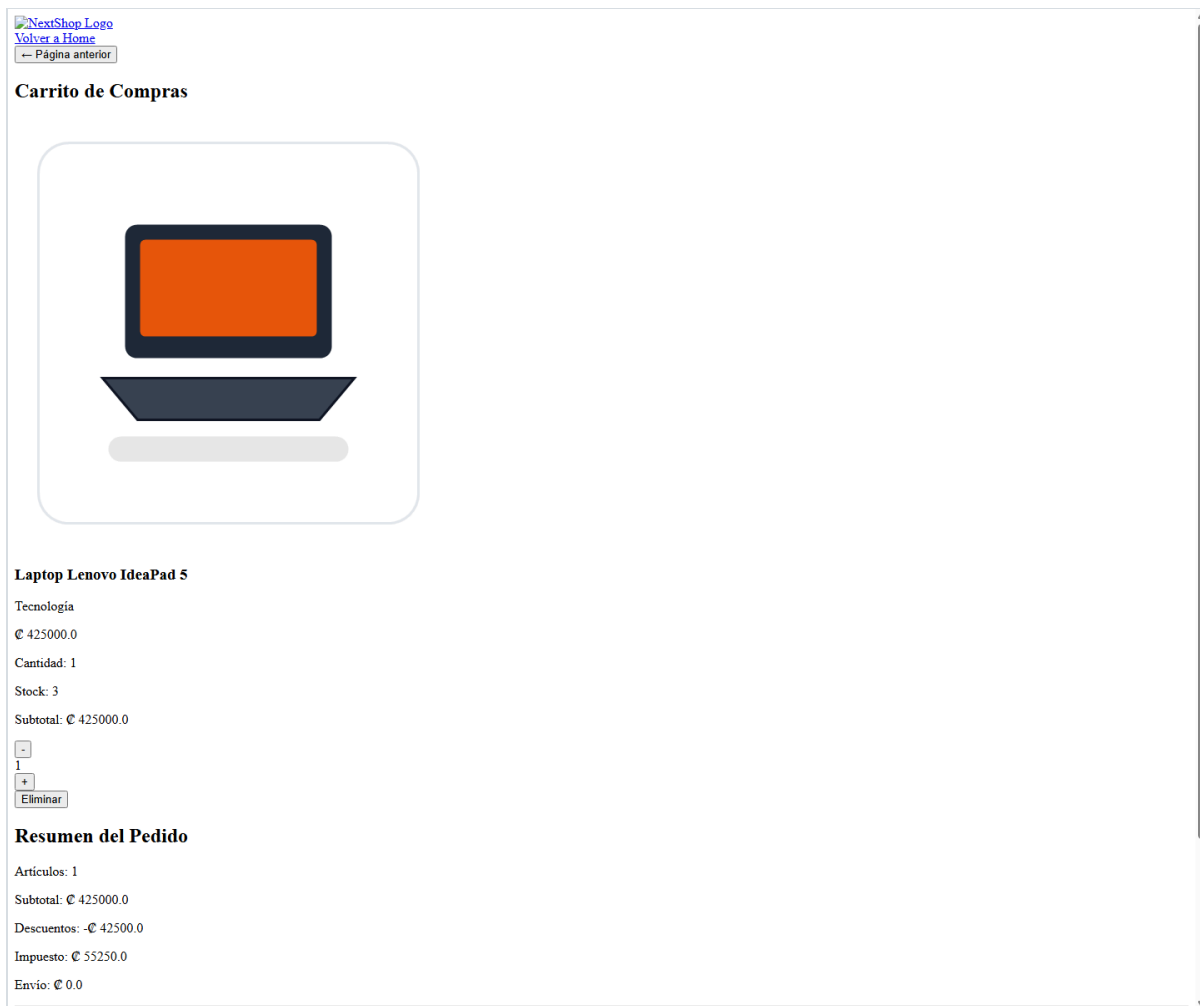
La funcionalidad se implementa principalmente con:

- `ShoppingCartController`
- `ShoppingCartService`
- `ShoppingCartRepository`
- `InventoryService`
- `ProductService`

El carrito muestra producto, cantidad, stock, subtotal y resumen del pedido. También utiliza `item.product.imagePath` para presentar la imagen del producto agregado.

Esta implementación corresponde al uso de formularios, solicitudes POST, redirecciones y sesión HTTP.

Evidencia



Carrito con imagen de producto

La captura muestra un producto agregado al carrito con su imagen. Evidencia la relación entre sesión de cliente, carrito e imagen del producto.

14. Módulo administrativo

El módulo administrativo permite consultar y mantener información del sistema. Incluye rutas como:

- /AdminProducts
- /AdminClients
- /AdminCategories
- /AdminDashboard

Desde estas pantallas se gestionan productos, clientes y categorías según las opciones implementadas. Las operaciones incluyen búsqueda, edición, activación y desactivación.

El acceso administrativo depende del perfil del usuario. Por esta razón, el informe documenta el módulo a partir de su implementación, sus rutas y las evidencias de control de versiones, sin alterar seguridad ni vistas para generar una captura artificial.

Esta implementación corresponde al control de acceso básico por perfil y al mantenimiento de registros desde formularios web.

Evidencia relacionada

- docs/evidencias/README.md : contiene el índice de evidencias visuales generadas para el informe.

15. Validaciones y reglas de negocio

NextShop aplica reglas de negocio desde controladores y servicios. Las principales son:

- No registrar clientes con correo duplicado.
- Validar correo, contraseña y estado del usuario al iniciar sesión.
- Diferenciar perfil cliente y perfil administrador.
- Evitar SKU duplicado al crear productos.
- Permitir activar o desactivar productos y usuarios.
- Validar existencia del producto antes de agregarlo al carrito.
- Validar stock disponible antes de agregar al carrito.
- Asociar el carrito al cliente autenticado.

Estas reglas reducen errores de uso y mantienen coherencia entre interfaz, datos y lógica.

Esta implementación corresponde a la validación del lado servidor estudiada en el curso.

16. Pruebas y evidencias

Las pruebas realizadas verifican que la aplicación compila, sirve recursos estáticos y renderiza productos con imágenes.

16.1 Pruebas funcionales

Prueba	Resultado	Evidencia
Home /	Correcto	01-home-productos-imagenes.png
Detalle /products/15	Correcto	02-detalle-producto-imagen.png

Imagen estática	HTTP 200	03-ruta-estatica-imagen.png
Carrito con producto	Correcto	04-carrito-imagen-producto.png

16.2 Pruebas técnicas

Verificación	Resultado
Productos con imagePath	99
Rutas únicas usadas	87
Duplicados con imagen compartida	12
Imágenes faltantes	0
Fallback conservado	productIcon.png

16.3 Compilación

El proyecto fue compilado con Maven:

```
mvn clean compile
```

Resultado esperado y obtenido:

```
BUILD SUCCESS
```

16.4 Evidencias visuales

Evidencia	Archivo	Qué demuestra
Home con productos	docs/evidencias/01-home-productos-imagenes.png	Renderizado del catálogo con imágenes
Detalle de producto	docs/evidencias/02-detalle-producto-imagen.png	Imagen asociada al producto seleccionado
Ruta estática	docs/evidencias/03-ruta-estatica-imagen.png	Spring Boot sirviendo PNG desde static/
Carrito	docs/evidencias/04-carrito-imagen-producto.png	Producto en carrito con imagen
ProductRepository	docs/evidencias/06-productrepository-imagepath.png	Rutas imagePath reales

Carpeta de imágenes	docs/evidencias/07-carpeta-static-images-products.png	Archivos PNG físicos disponibles
Build	docs/evidencias/08-build-success.png	Compilación exitosa
Git status	docs/evidencias/09-git-status.png	Estado del repositorio
Git log	docs/evidencias/10-git-log.png	Historial reciente de commits

Esta sección evidencia validación funcional, técnica y de control de versiones.

17. Control de versiones

El proyecto utiliza Git para registrar cambios y mantener trazabilidad. La rama documentada es:

```
integracion-edgardo-imagenes
```

Commit relevante:

```
34839c2 Conectar imagenes de productos y corregir template home
```

Este commit integra las rutas reales de imágenes en `ProductRepository` y corrige el nombre del template de home para evitar problemas en sistemas sensibles a mayúsculas y minúsculas.

El uso de Git respalda el trabajo colaborativo y permite auditar los cambios realizados.

Evidencias

git status

On branch integracion-edgardo-imagenes

Untracked files:

(use "git add <file>..." to include in what will be committed)

- docs/evidencias/01-home-productos-imagenes.png
- docs/evidencias/02-detalle-producto-imagen.png
- docs/evidencias/03-ruta-estatica-imagen.png
- docs/evidencias/04-carrito-imagen-producto.png
- docs/evidencias/06-productrepository-imagepath.png
- docs/evidencias/07-carpeta-static-images-products.png
- docs/evidencias/08-build-success.png
- docs/evidencias/09-git-status.png
- docs/evidencias/10-git-log.png

nothing added to commit but untracked files present (use "git add" to track)

Estado Git

La captura muestra el estado del repositorio durante la generación de evidencias. Es importante para respaldar la trazabilidad del trabajo documentado.

git log --oneline -5

```
34839c2 Conectar imagenes de productos y corregir template home
8e79928 Merge branch 'backup-imagenes-nextshop' into integracion-edgardo-imagenes
e7ca14e Backup avances catalogo e imagenes de productos
71f0e67 Resolución de bugs menores, y corrección en consulta principal de home, que estaba permitiendo ver productos desact
f8f12aa Incorporacion de cambio, método, calculos de items en carrito, impuesto, total, etc
```

Últimos commits

La captura muestra los commits recientes. Evidencia la trazabilidad del avance del módulo de imágenes y del fix de home.

18. Limitaciones actuales

La versión actual es funcional, pero conserva limitaciones propias de una etapa académica:

- Los datos se almacenan en memoria.
- La persistencia con base de datos aún no es la fuente principal.
- Las imágenes se manejan como archivos estáticos, no como BLOB.
- La seguridad se basa en sesión y perfil, no en una configuración completa de Spring Security.
- Algunas capturas administrativas no se generaron para evitar forzar autenticación o modificar vistas.
- En entornos Linux o Docker se debe mantener consistencia exacta en nombres de templates.

Estas limitaciones no impiden demostrar los conceptos principales del curso, pero sí marcan el camino para una versión más robusta.

19. Recomendaciones y trabajo futuro

Para futuras versiones se proponen las siguientes mejoras:

- Migrar repositorios en memoria a Spring Data JPA.
- Persistir productos, clientes, categorías, inventario y carrito en base de datos.
- Formalizar seguridad con Spring Security.
- Normalizar nombres de templates para despliegues en Linux o Docker.
- Mejorar la presentación visual del carrito y del panel administrativo.
- Agregar pruebas automatizadas para servicios y controladores.
- Mantener imágenes como archivos estáticos y guardar solo sus rutas en la base de datos.

Estas mejoras continúan la arquitectura actual sin requerir un rediseño completo.

20. Conclusiones

NextShop demuestra una aplicación web funcional construida con Spring Boot, Thymeleaf y una arquitectura MVC por capas. El proyecto integra navegación pública, detalle de productos, sesiones de usuario, carrito de compras y módulos administrativos.

La integración del catálogo visual fortalece la experiencia de usuario y evidencia el uso correcto de recursos estáticos. Cada producto del inventario queda conectado a una imagen mediante `imagePath`, manteniendo una separación clara entre datos y archivos físicos.

El proyecto también deja una base preparada para crecimiento: migración a JPA, mejora de seguridad, pruebas automatizadas y despliegue en un entorno más cercano a producción.

Esta implementación consolida los conceptos prácticos de Programación Web II: controladores, servicios, repositorios, vistas dinámicas, sesiones, validaciones y control de versiones.

Resumen ejecutivo de funcionalidades

Funcionalidad	Estado
Login	Implementado
Catálogo	Implementado
CRUD Productos	Implementado
Carrito	Implementado
Integración de imágenes	Implementado
Recursos estáticos	Implementado
Validaciones	Implementado
Build Maven	Correcto

21. Anexos

Anexo A: Documentos de apoyo

- docs/ProyectoProgramado.md
- docs/inventario-productos-imagenes.md
- docs/catalogo-visual-productos.md
- docs/fuente-imagenes-productos.md
- docs/catalogo-imagenes-productos.md

Anexo B: Evidencias visuales

- docs/evidencias/README.md
- docs/evidencias/01-home-productos-imagenes.png
- docs/evidencias/02-detalle-producto-imagen.png
- docs/evidencias/03-ruta-estatica-imagen.png
- docs/evidencias/04-carrito-imagen-producto.png

- docs/evidencias/06-productrepository-imagepath.png
- docs/evidencias/07-carpeta-static-images-products.png
- docs/evidencias/08-build-success.png
- docs/evidencias/09-git-status.png
- docs/evidencias/10-git-log.png

Anexo C: Datos del módulo de imágenes

Indicador	Valor
Registros de producto auditados	99
Rutas PNG únicas	87
Duplicados controlados	12
Imágenes faltantes	0
Fallback conservado	productIcon.png